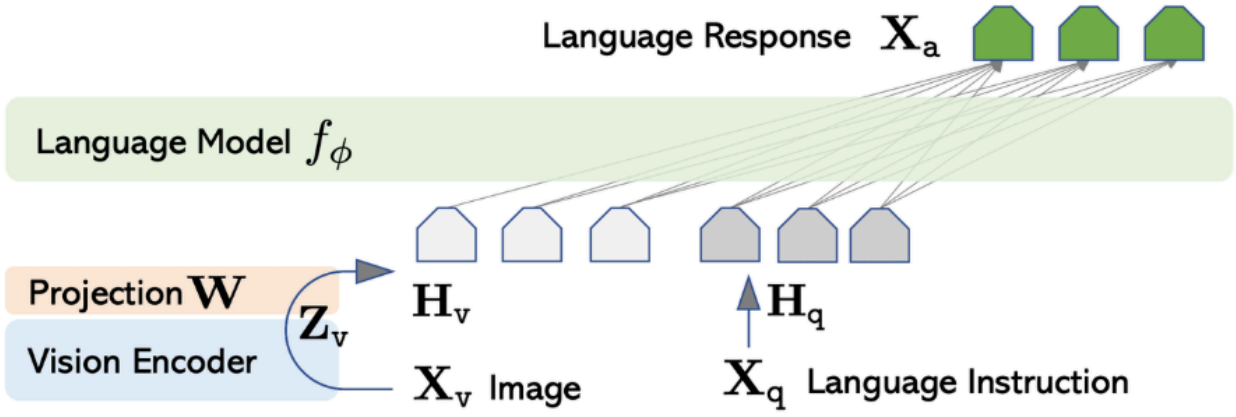
LLAVA（**Large Language-and-Vision Assistant**）

用最简单的架构（**ViT + Linear Layer + LLM**）证明了：只要数据质量够高，复杂的连接层是不必要的。

1.模型架构



Input=[Image]+[Text Prompt]

Visual Features=ViT(Image)

Image Token Embedding=LinearProj(Visual Features)

对齐图像与文本特征的维度

Output=LLM([Image Token Embedding,Text Embeddings])

Vicuna　　concat

2.两阶段训练

阶段一：特征对齐预训练

- ❌ 冻结 Vision Encoder (CLIP ViT)。
- ❌ 冻结 LLM (Vicuna/Llama)。
- ✅ 仅训练 Projector (Linear Layer)。

此时模型已经能将图像映射到文本空间

阶段二：多模态指令微调

- ❌ 冻结 Vision Encoder (可选解冻进行端到端微调，即 LLaVA-1.6 的做法)。
- ✅ 训练 Projector + 训练 LLM (通常全量微调或 LoRA)。

补充：指令微调(Instruction Tuning)

设计复杂的**<图片> + <指令/问题> + <详细回答>**，让模型学会**遵循指令、多轮对话、逻辑推理**。

LLaVA 成功的关键在于它使用了 **LLaVA-Instruct-150k** 数据集。这不是人工标注的，而是利用 **GPT-4** 生成的**合成数据**。

一条典型的指令微调数据包含三个部分：

1. `image`: 图片文件。
2. `conversations`: 一个列表，包含多轮对话（User 和 Assistant）。
3. **特殊 Token**: `<image>` 占位符。

```
{
  "id": "000000000001",
  "image": "coco/train2017/000000000001.jpg",
  "conversations": [
    {
      "from": "human",
      "value": "<image>\nwhat are the colors of the bus in the
image?"
    },
    {
```

```
      "from": "gpt",
      "value": "The bus in the image is primarily red with some
white details."
    },
    {
      "from": "human",
      "value": "Is it moving or parked?"
    },
    {
      "from": "gpt",
      "value": "Based on the blur of the wheels and the background,
it appears to be moving."
    }
  ]
}
```

数据不仅仅是描述，还包括：对话、推理、OCR、代码生成、数学计算等。

3.升级LLaVA

- **LLaVA1.5**

  模型结构上和前作相比，`LLaVA1.5` 将之前用于维度映射的的简单一层线性层替换为 `2` 层 线性层的 `MLP` 结构，并将 `clip-L/14` 的输入分辨率从 `224*224` 提升到 `336*336`，因为作者发现提高输入图像分辨率能够增强模型性能，LLM 换成了 Vicuna1.5（在 `LLama2` 上微调的模型）

- **LLaVA1.6（LLaVA-NeXT）**

  模型推理层面新的升级点在于，**Vision Encoder 支持更大的分辨率**，包括 672x672, 336x1344, 1344x336 几种分辨率的输入，并且支持通过图片裁切，编码，合并来实现，和前作一样的方法。毕竟，当提供高分辨率图像和保留细节的表征时，模型感知图像中复杂细节的能力会显著提高。它减少了面对低分辨率图像时的模型幻觉，即猜测想象的视觉内容。

4.核心伪代码

```
class CoreLLaVA:
    def __init__(self, vision_encoder, projector, llm, tokenizer):
        self.vision_encoder = vision_encoder
        self.projector = projector
        self.llm = llm
        self.tokenizer = tokenizer

    def encode_image(self, image):
        vision_feats = self.vision_encoder(image)   # [B, N, Dv]
        image_embeds = self.projector(vision_feats) # [B, N, Dl]
        return image_embeds

    def pack_embeddings(self, input_ids, image_embeds,
labels=None):
        text_embeds = self.llm.embed_tokens(input_ids)  # [T, Dl]

        packed = []
        packed_labels = []

        for i, tid in enumerate(input_ids):
            if tid == IMAGE_TOKEN_INDEX:
                for j in range(len(image_embeds)):
                    packed.append(image_embeds[j])
                    packed_labels.append(IGNORE_INDEX)
            else:
                packed.append(text_embeds[i])
                if labels is None:
                    packed_labels.append(IGNORE_INDEX)
                else:
                    packed_labels.append(labels[i])

        packed = stack(packed)  # [T', Dl]
        attention_mask = ones(len(packed))
        return packed, attention_mask, tensor(packed_labels)

    def forward(self, image, prompt_text, labels=None):
        image_embeds = self.encode_image(image)[0]   # 单样本时 [N,
Dl]
        input_ids = self.tokenizer(prompt_text)
```

```
        inputs_embeds, attention_mask, new_labels = \
            self.pack_embeddings(input_ids, image_embeds, labels)

        out = self.llm(
            inputs_embeds=inputs_embeds[None, ...],
            attention_mask=attention_mask[None, ...],
            labels=new_labels[None, ...] if labels is not None else
None
        )
        return out

    def generate(self, image, prompt_text, max_new_tokens=128):
        image_embeds = self.encode_image(image)[0]
        input_ids = self.tokenizer(prompt_text)

        inputs_embeds, attention_mask, _ = \
            self.pack_embeddings(input_ids, image_embeds,
labels=None)

        output_ids = self.llm.generate(
            inputs_embeds=inputs_embeds[None, ...],
            attention_mask=attention_mask[None, ...],
            max_new_tokens=max_new_tokens
        )
        return self.tokenizer.decode(output_ids[0])
```